

Topological Sort — Complete Placement + CP Guide

Target: SDE placements, online assessments, competitive programming

Covers: Topological Sort (DFS-based) · Kahn's Algorithm (BFS-based) · Course Schedule Problems · Cycle Detection in Directed Graphs · Longest Path in DAG · Lexicographically Smallest Topological Order

1. Topological Sorting (DFS-Based)

1. Overview

Topological sorting is a linear ordering of vertices in a **directed acyclic graph (DAG)** such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering. In other words, if there's a dependency from u to v , then u must appear before v .

A graph can have **multiple valid topological orderings**. The DFS-based approach uses the **finishing times** of nodes — a node is added to the ordering after all its descendants have been processed.

2. Intuition

Think of **prerequisites in a university course catalog**. You must take "Intro to Programming" before "Data Structures", and "Data Structures" before "Algorithms". A topological sort gives you a valid order to take all courses without violating prerequisites.

DFS-based approach:

- Imagine you're exploring a maze of dependencies. You start at a course and recursively explore all its prerequisites first.
- Once you've finished exploring a course's entire dependency chain, you "write it down" on a stack.
- The first course you finish writing down is one that has no dependents (it's a "leaf" dependency).
- In the end, reading the stack from top to bottom gives a valid order.

Why it works: In DFS, when we mark a node as "finished" (after processing all its neighbours), we know that all nodes that depend on it (its descendants) have already been finished. So adding it to the front of the result ensures the correct order.

3. When to Use It

- **Dependency resolution** — build systems, package managers, task scheduling
- **Course prerequisite ordering** — "Course Schedule" type problems
- **Compilation ordering** — determining which source files to compile first
- **Instruction scheduling** — ordering operations in compilers
- **Detecting cycles** in directed graphs (as a side effect)
- **Finding longest path** in DAG after topological ordering
- **Trigger phrases:** "prerequisites", "dependency order", "schedule courses", "build order", "linear ordering", "dependency graph", "task ordering"

4. When Not to Use It

- Graph has **cycles** — topological sort is only defined for DAGs
- Graph is **undirected** — topological sort is meaningless for undirected graphs
- Graph is **very dense and large** — Kahn's algorithm may be more intuitive
- Need **lexicographically smallest** order — use a priority queue with Kahn's algorithm instead
- Only need to **detect a cycle** — a simpler DFS with three-colour marking is enough without producing the sort

- **Overkill:** For a simple linear chain, just traverse the graph

5. Core Concepts

5.1 DAG (Directed Acyclic Graph)

- A directed graph with no cycles.
- **Why it matters:** Topological sort is only defined for DAGs. If a cycle exists, no valid linear ordering exists.

5.2 DFS Finishing Time (Post-order)

- The time at which a node's entire subtree has been explored.
- **Why it matters:** A node is added to the topological order **after** all its neighbours (dependencies) have been added. This is a reverse of DFS finishing order.

5.3 Visited States (Three-Colour)

- **White (0):** Unvisited
- **Grey (1):** In current DFS stack (being processed)
- **Black (2):** Fully processed
- **Why it matters:** The grey state helps detect cycles. If we encounter a grey node, a cycle exists.

5.4 Stack/Result Vector

- Stores the topological order. Nodes are pushed when they finish processing.
- **Why it matters:** The order of insertion (reverse of finishing) gives the correct linear order.

6. Step-by-Step Algorithm

1. Create an adjacency list `adj` for the graph with `n` vertices.
2. Create a `visited` array of size `n`, initialized to `0` (white/unvisited).
3. Create an empty stack `st` (or vector to store result).
4. For each vertex `u` from `0` to `n-1`:
 - If `visited[u] == 0`, call `dfs(u)`.
5. In `dfs(u)`:
 - Mark `visited[u] = 1` (grey — in current path).
 - For each neighbour `v` of `u`:
 - If `visited[v] == 1` → **cycle detected** (return false).
 - If `visited[v] == 0` → recursively call `dfs(v)`.
 - Mark `visited[u] = 2` (black — fully processed).
 - Push `u` onto the stack.
6. After all DFS calls, the stack (from top to bottom) gives the topological order.

7. Dry Run

Graph (DAG):

```

5 → 0 ← 4
↓       ↓
2 → 3 → 1

```

Edges: `5→0`, `5→2`, `4→0`, `4→1`, `2→3`, `3→1`

DFS starting from 0:

Step	Current Node	Visited State (0,1,2)	Stack (top → bottom)
Init	-	[0,0,0,0,0,0]	[]
1	dfs(0)	visited[0]=1 → no neighbours	[]
2	finish(0)	visited[0]=2	[0]
3	dfs(1)	visited[1]=1 → no neighbours	[0]
4	finish(1)	visited[1]=2	[0,1]
5	dfs(2)	visited[2]=1 → neighbour 3	[0,1]
6	dfs(3)	visited[3]=1 → neighbour 1 (black)	[0,1]
7	finish(3)	visited[3]=2	[0,1,3]
8	finish(2)	visited[2]=2	[0,1,3,2]
9	dfs(4)	visited[4]=1 → neighbours 0,1 (both black)	[0,1,3,2]
10	finish(4)	visited[4]=2	[0,1,3,2,4]
11	dfs(5)	visited[5]=1 → neighbours 0,2	[0,1,3,2,4]
12	finish(5)	visited[5]=2	[0,1,3,2,4,5]

Topological Order (stack top to bottom): 5 → 4 → 2 → 3 → 1 → 0

Verification: Every edge $u \rightarrow v$ has u before v in the order. ✓

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class TopologicalSortDFS {
public:
    // Returns topological order as a vector.
    // If graph has a cycle, returns empty vector.
    vector<int> topoSort(int n, vector<vector<int>> &adj) {
        vector<int> vis(n, 0);      // 0=white, 1=grey, 2=black
        vector<int> order;
        bool hasCycle = false;

        function<void(int)> dfs = [&](int u) {
            vis[u] = 1;              // mark grey (in current DFS path)
            for (int v : adj[u]) {
                if (vis[v] == 1) {   // back edge → cycle
                    hasCycle = true;
                    return;
                }
                if (vis[v] == 0) {
                    dfs(v);
                    if (hasCycle) return;
                }
            }
            vis[u] = 2;              // mark black (fully processed)
            order.push_back(u);      // add to order after children
        };
```

```

        for (int i = 0; i < n; i++) {
            if (vis[i] == 0) {
                dfs(i);
                if (hasCycle) return {};    // cycle detected → no valid order
            }
        }

        reverse(order.begin(), order.end()); // reverse to get correct order
        return order;
    }
};

int main() {
    int n = 6;
    vector<vector<int>> adj(n);
    adj[5] = {0, 2};
    adj[4] = {0, 1};
    adj[2] = {3};
    adj[3] = {1};

    TopologicalSortDFS solver;
    vector<int> order = solver.topoSort(n, adj);

    if (order.empty()) {
        cout << "Graph has a cycle. No valid topological order.\n";
    } else {
        cout << "Topological Order: ";
        for (int v : order) cout << v << " ";
        cout << "\n";
    }
    return 0;
}

```

Output:

```
Topological Order: 5 4 2 3 1 0
```

Python Implementation

```

from typing import List

class TopologicalSortDFS:
    @staticmethod
    def topo_sort(n: int, adj: List[List[int]]) -> List[int]:
        vis = [0] * n          # 0=white, 1=grey, 2=black
        order = []
        has_cycle = False

        def dfs(u: int) -> None:
            nonlocal has_cycle
            vis[u] = 1          # mark grey
            for v in adj[u]:
                if vis[v] == 1:  # back edge → cycle
                    has_cycle = True
            return

        for i in range(n):
            if vis[i] == 0:
                dfs(i)
                if has_cycle:
                    return []

        order = order[::-1]
        return order

```

```

        if vis[v] == 0:
            dfs(v)
            if has_cycle:
                return

        vis[u] = 2                # mark black
        order.append(u)          # post-order

    for i in range(n):
        if vis[i] == 0:
            dfs(i)
            if has_cycle:
                return []        # cycle detected

    return order[::-1]           # reverse to get correct order

if __name__ == "__main__":
    n = 6
    adj = [[] for _ in range(n)]
    adj[5] = [0, 2]
    adj[4] = [0, 1]
    adj[2] = [3]
    adj[3] = [1]

    order = TopologicalSortDFS.topo_sort(n, adj)
    if not order:
        print("Graph has a cycle. No valid topological order.")
    else:
        print("Topological Order:", order)

```

10. Code Explanation

DFS-based Topological Sort:

- vis array (three-colour):** We use three states: 0 (unvisited), 1 (in current DFS stack), 2 (finished). This is crucial for cycle detection. If we encounter a node with state 1 during DFS, it means we've found a back edge, indicating a cycle.
- Lambda dfs:** A recursive function that processes a node. It marks the node as grey, explores all neighbours, and only marks it black (and adds to order) after all neighbours are done.
- Cycle detection:** If `vis[v] == 1`, we've found a back edge (an edge to a node currently in the call stack). This means the graph has a cycle, and topological sort is impossible.
- Post-order insertion:** The node is added to `order` after processing all its neighbours. This ensures that dependencies come after the dependent in the list (before reversing).
- Reverse:** We reverse at the end because a node is added when it finishes (after its dependencies). The first finished node should be last in the topological order. So we reverse.
- Graph traversal:** We call `dfs` for every unvisited node to handle disconnected graphs.

11. Complexity Analysis

Operation	Time	Space
DFS-based Topological Sort	$O(V + E)$	$O(V)$ for recursion stack + visited array + order
Overall	$O(V + E)$	$O(V)$

- **V** = number of vertices, **E** = number of edges
- Each vertex and each edge is visited exactly once.
- Space includes the recursion stack (up to $O(V)$ in worst case) and the visited/order arrays.

12. Common Patterns

12.1 Standard Topological Order

- **How to identify:** "Find a linear ordering", "dependency resolution", "build order"
- **Approach:** Use DFS-based topological sort.
- **Example:** [LeetCode 210 — Course Schedule II](#)

12.2 Cycle Detection in Directed Graph

- **How to identify:** "Detect cycle in directed graph", "prerequisite cycle"
- **Approach:** Use three-colour DFS. If a back edge is found, a cycle exists.
- **Example:** [LeetCode 207 — Course Schedule](#)

12.3 All Possible Topological Orders

- **How to identify:** "Number of ways to order", "all valid orders"
- **Approach:** Backtracking with Kahn's algorithm (trying all possible sources at each step).
- **Example:** [GFG — All Topological Sorts of a DAG](#)

13. Common Mistakes

- **Forgetting to reverse:** The post-order list is in reverse topological order. Forgetting to reverse produces the wrong order.
- **Using only two states:** Using only visited/unvisited fails to detect back edges. You need three states (white/gray/black) for cycle detection in directed graphs.
- **Not handling disconnected graphs:** Running DFS from only one source misses nodes in other components.
- **Assuming unique order:** A DAG can have multiple valid topological orders. This is not a bug.
- **Stack overflow on large graphs:** Recursive DFS may overflow the stack for very deep graphs (e.g., 10^5 nodes). Use iterative DFS or Kahn's algorithm.
- **Modifying graph during traversal:** Do not modify the adjacency list while traversing.

14. Edge Cases

- Empty graph ($n=0$): Return empty list.
- Single node ($n=1$): Return `[0]`.
- Line graph ($1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow n$): Exactly one valid order.
- Disconnected DAG: Multiple components, order is still valid.
- Graph with self-loop (cycle of length 1): Detect cycle immediately.
- Complete DAG: Every pair of nodes has a directed edge in one direction. Only one valid order.
- Reverse graph: If all edges are reversed, topological order is reversed too.

15. Variations

15.1 Iterative DFS (Stack-based)

- Uses explicit stack instead of recursion.
- **When to use:** When recursion depth may exceed the stack limit (e.g., 10^5+ nodes in CP).
- **Placement relevance:** Moderate. Interviewers usually accept recursive DFS.

15.2 Lexicographically Smallest Topological Order

- Find the smallest lexicographic order among all valid orders.
- **When to use:** When the problem explicitly asks for the smallest order.
- **Implementation:** Use Kahn's algorithm with a min-heap instead of a queue.
- **Placement relevance:** High. Common in CP and some OAs.

16. Related Algorithms/Data Structures

- **Kahn's Algorithm (BFS-based):** Alternative approach for topological sort. Also detects cycles. More intuitive and avoids recursion.
- **DFS Cycle Detection:** A simpler version of the same idea without building the topological order.
- **Strongly Connected Components (Kosaraju/Tarjan):** Topological sort of the condensation DAG of SCCs.
- **Longest Path in DAG:** Uses topological sort as a preprocessing step.
- **Shortest Path in DAG:** Also uses topological sort + relaxation.

2. Kahn's Algorithm (BFS-Based Topological Sort)

1. Overview

Kahn's algorithm is a **BFS-based** approach to topological sorting. Instead of using DFS finishing times, it works by **removing nodes with no incoming edges (in-degree = 0)** one by one. It is more intuitive and naturally handles cycle detection without needing three-colour marking.

2. Intuition

Imagine a **building construction project**. You have tasks like "lay foundation", "build walls", "install roof", "do wiring", "plumbing". Some tasks depend on others.

The approach:

- Start with all tasks that have **no prerequisites** (in-degree = 0). These can be done immediately.
- After completing a task, remove it from the graph. This may reduce the prerequisites count for other tasks.
- If a task now has no remaining prerequisites, add it to the queue.
- Continue until all tasks are processed or no tasks with in-degree 0 remain.

Why it works: A topological order exists iff the graph is a DAG. By always removing nodes with no incoming edges, we ensure we never process a node before its dependencies. If at some point no node has in-degree 0 but nodes remain unprocessed, a cycle exists.

3. When to Use It

- Same as DFS-based topological sort — any DAG ordering problem
- When you need **lexicographically smallest** order (use min-heap instead of queue)
- When you want to **avoid recursion** (iterative, stack-safe)
- When you need to **process in dependency order** (like a build system scheduler)
- **Course Schedule** problems (LeetCode 207, 210)
- **Trigger phrases:** same as DFS-based + "in-degree", "source removal", "process in order of dependencies"

4. When Not to Use It

- Graph is **undirected** — topological sort is not defined
- You need to detect **which nodes are in a cycle** (DFS-based approach is better for this)
- Graph is **very sparse** and recursion depth is safe — DFS may be simpler to code
- **Overkill:** For a simple linear chain, both approaches work fine

5. Core Concepts

5.1 In-Degree

- The number of incoming edges to a vertex.
- Why it matters:** A node with in-degree 0 has no dependencies and can be placed first in the order.

5.2 Queue (or Min-Heap)

- Stores nodes with in-degree 0, ready to be processed.
- Why it matters:** Using a standard queue gives any valid order. Using a min-heap gives the lexicographically smallest order.

5.3 In-Degree Update

- When a node is processed, decrement the in-degree of all its neighbours.
- Why it matters:** This simulates "removing" the node from the graph and updating dependency counts.

5.4 Cycle Detection

- If the number of processed nodes < total nodes, a cycle exists.
- Why it matters:** In a cycle, no node ever has in-degree 0, so those nodes are never processed.

6. Step-by-Step Algorithm

- Create adjacency list `adj` for the graph.
- Compute `indegree[v]` for every vertex `v` (count of incoming edges).
- Create a queue `q` (or min-heap for lexicographically smallest).
- Push all vertices with `indegree == 0` into the queue.
- Initialize `count = 0` (number of processed nodes) and `order = []`.
- While `q` is not empty:
 - Pop front node `u`.
 - Add `u` to `order`.
 - `count++`.
 - For each neighbour `v` of `u`:
 - Decrement `indegree[v]`.
 - If `indegree[v] == 0`, push `v` into `q`.
- If `count != n`, the graph has a cycle → no valid topological order.

7. Dry Run

Graph (same as before):

```

5 → 0 ← 4
↓       ↓
2 → 3 → 1

```

In-degree array: `indeg = [2, 2, 1, 1, 0, 0]` (nodes 0 has 2 incoming edges, 1 has 2, etc.)

Step	Queue (front→back)	Processed	indeg[0]	indeg[1]	indeg[2]	indeg[3]	indeg[4]	indeg[5]	Order
Init	[4,5]	0	2	2	1	1	0	0	[]

Step	Queue (front→back)	Processed	indeg[0]	indeg[1]	indeg[2]	indeg[3]	indeg[4]	indeg[5]	Order
1	Pop 4 → [5]	1	2→1	2→1	1	1	0	0	[4]
2	Pop 5 → []	2	1→0	1	1→0	1	0	0	[4,5]
3	Push 0 → [0]	2	0	1	0	1	0	0	[4,5]
4	Pop 0 → []	3	0	1	0	1	0	0	[4,5,0]
5	Push 2 → [2]	3	0	1	0	1	0	0	[4,5,0]
6	Pop 2 → []	4	0	1	0	1→0	0	0	[4,5,0,2]
7	Push 3 → [3]	4	0	1	0	0	0	0	[4,5,0,2]
8	Pop 3 → []	5	0	1→0	0	0	0	0	[4,5,0,2,3]
9	Push 1 → [1]	5	0	0	0	0	0	0	[4,5,0,2,3]
10	Pop 1 → []	6	0	0	0	0	0	0	[4,5,0,2,3,1]

Topological Order: [4, 5, 0, 2, 3, 1]

Note: This is a different valid order than DFS (which gave [5, 4, 2, 3, 1, 0]). Both are valid.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class KahnTopologicalSort {
public:
    // Returns topological order. If cycle exists, returns empty vector.
    vector<int> topoSort(int n, vector<vector<int>> &adj) {
        vector<int> indeg(n, 0);
        // Compute in-degree for each vertex
        for (int u = 0; u < n; u++) {
            for (int v : adj[u]) {
                indeg[v]++;
            }
        }

        queue<int> q;
        // Push all vertices with in-degree 0
        for (int i = 0; i < n; i++) {
            if (indeg[i] == 0) q.push(i);
        }

        vector<int> order;
        int processed = 0;

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            order.push_back(u);
            processed++;

            for (int v : adj[u]) {
                indeg[v]--;
            }
        }

        return order;
    }
};
```

```

        if (indeg[v] == 0) q.push(v);
    }
}

if (processed != n) return {}; // cycle detected
return order;
}

// Lexicographically smallest topological order
vector<int> topoSortLexicographically(int n, vector<vector<int>> &adj) {
    vector<int> indeg(n, 0);
    for (int u = 0; u < n; u++) {
        for (int v : adj[u]) {
            indeg[v]++;
        }
    }

    priority_queue<int, vector<int>, greater<int>> pq; // min-heap
    for (int i = 0; i < n; i++) {
        if (indeg[i] == 0) pq.push(i);
    }

    vector<int> order;
    int processed = 0;

    while (!pq.empty()) {
        int u = pq.top();
        pq.pop();
        order.push_back(u);
        processed++;

        for (int v : adj[u]) {
            indeg[v]--;
            if (indeg[v] == 0) pq.push(v);
        }
    }

    if (processed != n) return {}; // cycle detected
    return order;
}

int main() {
    int n = 6;
    vector<vector<int>> adj(n);
    adj[5] = {0, 2};
    adj[4] = {0, 1};
    adj[2] = {3};
    adj[3] = {1};

    KahnTopologicalSort solver;
    vector<int> order = solver.topoSort(n, adj);

    if (order.empty()) {
        cout << "Graph has a cycle.\n";
    } else {
        cout << "Kahn's Topological Order: ";
        for (int v : order) cout << v << " ";
        cout << "\n";
    }
}

```

```
// Lexicographically smallest
vector<int> lexOrder = solver.topoSortLexicographically(n, adj);
cout << "Lexicographically Smallest: ";
for (int v : lexOrder) cout << v << " ";
cout << "\n";

return 0;
}
```

Output:

```
Kahn's Topological Order: 4 5 0 2 3 1
Lexicographically Smallest: 4 5 0 2 3 1
```

Python Implementation

```
from collections import deque, defaultdict
from typing import List
import heapq

class KahnTopologicalSort:
    @staticmethod
    def topo_sort(n: int, adj: List[List[int]]) -> List[int]:
        indeg = [0] * n
        for u in range(n):
            for v in adj[u]:
                indeg[v] += 1

        q = deque([i for i in range(n) if indeg[i] == 0])
        order = []
        processed = 0

        while q:
            u = q.popleft()
            order.append(u)
            processed += 1
            for v in adj[u]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    q.append(v)

        return order if processed == n else []

    @staticmethod
    def topo_sort_lexicographically(n: int, adj: List[List[int]]) -> List[int]:
        indeg = [0] * n
        for u in range(n):
            for v in adj[u]:
                indeg[v] += 1

        pq = [i for i in range(n) if indeg[i] == 0]
        heapq.heapify(pq)
        order = []
        processed = 0
```

```

while pq:
    u = heapq.heappop(pq)
    order.append(u)
    processed += 1
    for v in adj[u]:
        indeg[v] -= 1
        if indeg[v] == 0:
            heapq.heappush(pq, v)

return order if processed == n else []

if __name__ == "__main__":
    n = 6
    adj = [[] for _ in range(n)]
    adj[5] = [0, 2]
    adj[4] = [0, 1]
    adj[2] = [3]
    adj[3] = [1]

    order = KahnTopologicalSort.topo_sort(n, adj)
    print("Kahn's Topological Order:", order)

    lex_order = KahnTopologicalSort.topo_sort_lexicographically(n, adj)
    print("Lexicographically Smallest:", lex_order)

```

10. Code Explanation

Kahn's Algorithm:

- In-degree computation:** We iterate through all edges and count incoming edges for each vertex. This is $O(V+E)$.
- Queue initialization:** All vertices with in-degree 0 have no dependencies and can be processed first. They form the starting set.
- Main loop:** We pop a node, add it to the order, and for each neighbour, decrement its in-degree. If a neighbour's in-degree becomes 0, it means all its dependencies are now satisfied, so we add it to the queue.
- Cycle detection:** If `processed != n`, some nodes never reached in-degree 0 because they're part of a cycle. In a cycle, each node has at least one incoming edge from within the cycle, so its in-degree never drops to 0.
- Lexicographically smallest:** By using a min-heap instead of a queue, we always pick the smallest-indexed node among those with in-degree 0. This gives the lexicographically smallest valid order.

11. Complexity Analysis

Operation	Time	Space
Kahn's Algorithm (standard)	$O(V + E)$	$O(V)$
Kahn's Algorithm (lexicographically smallest)	$O(V + E + V \log V) = O(V \log V + E)$	$O(V)$
Overall	$O(V + E)$	$O(V)$

- V** = number of vertices, **E** = number of edges
- The priority queue version adds $O(\log V)$ per pop/push, making it $O(V \log V + E)$.

12. Common Patterns

12.1 Standard Topological Sort (Kahn's)

- **How to identify:** Dependency ordering, task scheduling
- **Approach:** Use queue, track in-degrees, check processed count.
- **Example:** [LeetCode 210 — Course Schedule II](#)

12.2 Lexicographically Smallest Order

- **How to identify:** "Smallest order", "minimum order", "dictionary order"
- **Approach:** Use min-heap instead of queue.
- **Example:** [Codeforces — Fox And Names](#)

12.3 Is Graph a DAG? (Cycle Detection)

- **How to identify:** "Is it possible to schedule?", "detect cycle in directed graph"
- **Approach:** Run Kahn's algorithm. If processed $< n$, a cycle exists.
- **Example:** [LeetCode 207 — Course Schedule](#)

13. Common Mistakes

- **Not computing in-degree correctly:** Forgetting to count all incoming edges from all vertices.
- **Modifying in-degree while iterating:** Use a separate array, not a copy by reference.
- **Not checking processed count:** Always check if `processed == n` to detect cycles.
- **Using DFS for Kahn's:** Kahn's is BFS-based; using a stack instead of queue gives a different (possibly invalid) order.
- **Incorrect graph construction:** For directed graphs, ensure edges go from dependency to dependent.
- **Overflow:** Not an issue for topological sort, but in-degree can be up to $V-1$, which fits in int.

14. Edge Cases

- **Graph with no edges:** All nodes have in-degree 0. Any order is valid.
- **Single node with self-loop:** `indeg[0] = 1`, never pushed to queue. Processed = 0, cycle detected.
- **Disconnected DAG:** Multiple components, each component's nodes are processed independently.
- **All nodes in a cycle:** No node has in-degree 0 initially. Queue is empty. Cycle detected immediately.
- **Multiple sources:** Several nodes with in-degree 0. The order among them depends on queue order.

15. Variations

15.1 Kahn's with Reverse Graph (Topological Sort of Reversed DAG)

- Reverse all edges and run Kahn's algorithm.
- **When to use:** When the problem asks for "reverse dependencies" or "what depends on X".
- **Placement relevance:** Moderate.

15.2 Minimum Number of Semesters (Course Schedule III)

- Variant where each node (course) takes one unit of time and you can take at most k courses per semester.
- **When to use:** Scheduling with capacity constraints.
- **Approach:** Kahn's algorithm with level-by-level processing.
- **Placement relevance:** High. Common in OAs.

16. Related Algorithms/Data Structures

- **DFS-based Topological Sort:** The other approach. Neither is strictly better — DFS is easier to code, Kahn's is more intuitive.

- **BFS traversal:** Kahn's algorithm is essentially BFS on the in-degree-based graph.
 - **Minimum Spanning Tree:** Not directly related, but both deal with ordering constraints.
 - **Shortest Path in DAG:** Uses topological order (from either method) as a preprocessing step.
-

3. Course Schedule Problems

1. Overview

Course Schedule problems are a classic application of topological sort. Given n courses labeled 0 to $n-1$ and a list of prerequisite pairs $[a, b]$ meaning "you must take course b before course a ", determine if it's possible to complete all courses (cycle detection) or find a valid order (topological sort).

These problems appear in almost every major placement drive and interview round.

2. Intuition

Think of each course as a node in a directed graph. An edge $b \rightarrow a$ means "b must be taken before a" (or equivalently, "a depends on b").

Two variants:

- **Course Schedule I (LeetCode 207):** Just detect if a cycle exists. Can you finish all courses?
- **Course Schedule II (LeetCode 210):** Return a valid order. If impossible, return empty.

The intuition is the same as topological sort — if the prerequisite graph has a cycle, it's impossible to complete all courses.

3. When to Use It

- **Any problem with prerequisites** — courses, tasks, modules
- **Dependency resolution** — "Can all tasks be completed?"
- **Build systems** — "In what order should we compile these files?"
- **Trigger phrases:** "prerequisites", "course schedule", "before taking", "dependency", "require", "complete all tasks"

4. When Not to Use It

- **No dependencies** — just return any order (or `true`)
- **Dependencies are not acyclic** — the problem requires detecting impossibility, which is exactly what we do
- **Not a pure DAG problem** — if there are additional constraints (time, capacity, etc.), it's a variant

5. Core Concepts

5.1 Graph Construction

- For each prerequisite $[a, b]$ (b must be taken before a), add edge $b \rightarrow a$.
- **Why it matters:** Correct edge direction is critical. Getting it reversed produces wrong results.

5.2 Cycle Detection

- A cycle means it's impossible to complete all courses.
- **Why it matters:** This is the core of Course Schedule I.

5.3 Topological Order

- A valid sequence of courses respecting all prerequisites.
- **Why it matters:** This is the core of Course Schedule II.

6. Step-by-Step Algorithm

Course Schedule I (Cycle Detection):

1. Build adjacency list from prerequisites.
2. Compute in-degree for each course.
3. Push all courses with in-degree 0 into a queue.
4. Process using Kahn's algorithm.
5. If processed == n, return true; else return false (cycle exists).

Course Schedule II (Find Order):

1. Same as above, but also collect the order.
2. If processed == n, return order; else return empty.

7. Dry Run

Input: $n = 4$, $\text{prerequisites} = [[1,0],[2,0],[3,1],[3,2]]$

Graph: $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 3, 2 \rightarrow 3$

In-degree: $\text{indeg} = [0, 1, 1, 2]$

Step	Queue	Processed	Order
Init	[0]	0	[]
1	Pop 0 $\rightarrow$ push 1,2	1	[0]
2	Pop 1 $\rightarrow$ push 3	2	[0,1]
3	Pop 2 $\rightarrow$ push (3 already in queue)	3	[0,1,2]
4	Pop 3	4	[0,1,2,3]

Result: Possible. Order: $[0, 1, 2, 3]$.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class CourseSchedule {
public:
    // Course Schedule I: Can finish all courses?
    bool canFinish(int n, vector<vector<int>> &prerequisites) {
        vector<vector<int>> adj(n);
        vector<int> indeg(n, 0);

        // Build graph: prerequisite [a, b] means "take b before a"
        // Edge: b  $\rightarrow$  a
        for (auto &p : prerequisites) {
            int a = p[0], b = p[1];
            adj[b].push_back(a);
            indeg[a]++;
        }

        queue<int> q;
        for (int i = 0; i < n; i++) {
```

```

        if (indeg[i] == 0) q.push(i);
    }

    int processed = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        processed++;
        for (int v : adj[u]) {
            indeg[v]--;
            if (indeg[v] == 0) q.push(v);
        }
    }

    return processed == n;
}

// Course Schedule II: Return a valid order
vector<int> findOrder(int n, vector<vector<int>> &prerequisites) {
    vector<vector<int>> adj(n);
    vector<int> indeg(n, 0);

    for (auto &p : prerequisites) {
        int a = p[0], b = p[1];
        adj[b].push_back(a);
        indeg[a]++;
    }

    queue<int> q;
    for (int i = 0; i < n; i++) {
        if (indeg[i] == 0) q.push(i);
    }

    vector<int> order;
    int processed = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        processed++;
        for (int v : adj[u]) {
            indeg[v]--;
            if (indeg[v] == 0) q.push(v);
        }
    }

    if (processed != n) return {}; // cycle → impossible
    return order;
}

};

int main() {
    CourseSchedule cs;

    // Test: Can finish?
    vector<vector<int>> prereq = {{1,0},{2,0},{3,1},{3,2}};
    cout << "Can finish (n=4): " << (cs.canFinish(4, prereq) ? "Yes" : "No") << "\n";

    // Test: Find order
    vector<int> order = cs.findOrder(4, prereq);

```

```

if (order.empty()) {
    cout << "Impossible to schedule.\n";
} else {
    cout << "Order: ";
    for (int c : order) cout << c << " ";
    cout << "\n";
}

// Test with cycle
vector<vector<int>> cyclePrereq = {{1,0},{0,1}};
cout << "Can finish (cycle, n=2): " << (cs.canFinish(2, cyclePrereq) ? "Yes" : "No") <<
"\n";

return 0;
}

```

Output:

```

Can finish (n=4): Yes
Order: 0 1 2 3
Can finish (cycle, n=2): No

```

Python Implementation

```

from collections import deque
from typing import List

class CourseSchedule:
    @staticmethod
    def can_finish(n: int, prerequisites: List[List[int]]) -> bool:
        adj = [[] for _ in range(n)]
        indeg = [0] * n

        for a, b in prerequisites:
            adj[b].append(a)
            indeg[a] += 1

        q = deque([i for i in range(n) if indeg[i] == 0])
        processed = 0

        while q:
            u = q.popleft()
            processed += 1
            for v in adj[u]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    q.append(v)

        return processed == n

    @staticmethod
    def find_order(n: int, prerequisites: List[List[int]]) -> List[int]:
        adj = [[] for _ in range(n)]
        indeg = [0] * n

```

```

    for a, b in prerequisites:
        adj[b].append(a)
        indeg[a] += 1

    q = deque([i for i in range(n) if indeg[i] == 0])
    order = []
    processed = 0

    while q:
        u = q.popleft()
        order.append(u)
        processed += 1
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)

    return order if processed == n else []

if __name__ == "__main__":
    prereq = [[1, 0], [2, 0], [3, 1], [3, 2]]
    print("Can finish:", CourseSchedule.can_finish(4, prereq))
    print("Order:", CourseSchedule.find_order(4, prereq))

    cycle_prereq = [[1, 0], [0, 1]]
    print("Can finish (cycle):", CourseSchedule.can_finish(2, cycle_prereq))

```

10. Code Explanation

- Graph construction:** For each prerequisite $[a, b]$, we add an edge $b \rightarrow a$ because b must be taken before a . This is the standard convention in Course Schedule problems.
- In-degree tracking:** $\text{indeg}[a]++$ because a has one more prerequisite.
- Kahn's algorithm:** Exactly the same as the standard implementation. We process nodes with in-degree 0.
- Result:** For `canFinish`, we return `processed == n`. For `findOrder`, we return the order or empty.
- Edge direction:** Some problems define $[a, b]$ as "a depends on b" (a must be taken after b). Always check the problem statement carefully.

11. Complexity Analysis

Operation	Time	Space
Graph construction	$O(P)$ where P = number of prerequisites	$O(n + P)$ for adjacency list
Kahn's algorithm	$O(n + P)$	$O(n)$
Overall	$O(n + P)$	$O(n + P)$

12. Common Patterns

12.1 Basic Course Schedule

- How to identify:** "Can you finish all courses?" with prerequisite pairs.
- Approach:** Kahn's algorithm, check processed count.

- **Example:** [LeetCode 207 — Course Schedule](#)

12.2 Course Schedule with Order

- **How to identify:** "Return a valid order of courses."
- **Approach:** Kahn's algorithm, collect order.
- **Example:** [LeetCode 210 — Course Schedule II](#)

12.3 Parallel Courses (Minimum Semesters)

- **How to identify:** "Minimum number of semesters to take all courses", "can take k courses per semester"
- **Approach:** Level-by-level Kahn's algorithm (BFS layers). Each level is one semester.
- **Example:** [LeetCode 1136 — Parallel Courses](#)

12.4 Course Schedule with Capacity

- **How to identify:** "Maximum k courses per semester", "minimum time to complete"
- **Approach:** Level-by-level Kahn's with a priority queue to prioritize courses with more dependents.
- **Example:** [LeetCode 2050 — Parallel Courses III](#)

13. Common Mistakes

- **Wrong edge direction:** The most common mistake. Read the problem carefully: is `[a, b]` meaning "a must be taken before b" or "b must be taken before a"?
- **Not handling disconnected graphs:** A course with no prerequisites should have in-degree 0 and be processed.
- **Assuming unique order:** If multiple orders exist, any valid one is acceptable.
- **Forgetting to detect cycles:** Always check `processed == n`.
- **Using DFS with two states:** For cycle detection, you need three states. Kahn's is simpler for Course Schedule.

14. Edge Cases

- **n = 0:** Return true / empty list.
- **n = 1, no prerequisites:** Return true / [0].
- **All courses independent:** Any order works.
- **Single chain:** One valid order, straightforward.
- **Self-loop:** `[0, 0] → impossible`.
- **Multiple disconnected cycles:** Impossible.
- **Large n (10⁵):** Kahn's algorithm works fine. Avoid recursive DFS.

15. Variations

15.1 Minimum Number of Semesters

- Each course takes one semester, you can take unlimited courses per semester.
- **Approach:** Level-by-level BFS (Kahn's). Each level is one semester.
- **Placement relevance:** High.

15.2 Course Schedule with Time (Parallel Courses III)

- Each course has a duration, and you have unlimited parallelism.
- **Approach:** Topological sort + DP. The earliest time to complete a course is `max(earliest completion of prerequisites) + duration`.
- **Placement relevance:** High. Common in OAs.

15.3 Alien Dictionary (Topological Sort of Letters)

- Given a sorted dictionary of an alien language, find the order of letters.
- **Approach:** Compare adjacent words to extract character ordering constraints, then topological sort.
- **Placement relevance:** High. Very common in interviews.

16. Related Algorithms/Data Structures

- **Topological Sort (DFS):** Alternative implementation.
- **Kahn's Algorithm:** Preferred for Course Schedule problems.
- **DP on DAG:** Used for Parallel Courses III (time-based).
- **Union-Find:** Not suitable for directed graphs. Don't use DSU for Course Schedule.

4. Cycle Detection Using Topological Sort

1. Overview

A directed graph has a cycle if and only if topological sort fails. Both DFS-based and Kahn's algorithm approaches can detect cycles in directed graphs. This is one of the most important applications of topological sort.

2. Intuition

Kahn's approach: In a DAG, every node has a finite number of dependencies. If we keep removing nodes with no dependencies, we'll eventually remove all nodes. If some nodes remain, they must be in a cycle — each node in the cycle has at least one incoming edge from within the cycle, so its in-degree never reaches 0.

DFS approach: When performing DFS, if we encounter a node that is currently in the recursion stack (a "back edge"), we've found a cycle. Think of it like being in a maze and finding a path that leads back to a room you're currently exploring — you're going in circles.

3. When to Use It

- **Any directed graph cycle detection problem**
- **Validating DAGs** before applying topological sort or DP
- **Deadlock detection** in operating systems
- **Prerequisite validation** in course scheduling
- **Trigger phrases:** "detect cycle", "graph has a cycle", "is it a DAG", "prerequisite cycle"

4. When Not to Use It

- **Undirected graph** — use DFS with parent tracking or DSU instead
- **Only need to detect if a cycle exists in a DAG you know is acyclic** — unnecessary
- **Graph is very small** — a simple DFS with a visited set is enough
- **Overkill:** For a simple reachability check, just use BFS/DFS

5. Core Concepts

5.1 Back Edge

- An edge from a node to an ancestor in the DFS tree.
- **Why it matters:** A back edge is the defining characteristic of a cycle in a directed graph.

5.2 Three-Colour Marking (DFS)

- White (0): Not visited. Grey (1): In current DFS recursion stack. Black (2): Fully processed.
- **Why it matters:** If we find an edge to a grey node, we've found a cycle. Black nodes are safe.

5.3 Processed Count (Kahn's)

- Number of nodes that have been added to the topological order.
- **Why it matters:** If `processed < n` after running Kahn's, a cycle exists.

6. Step-by-Step Algorithm

Kahn's Approach:

1. Build adjacency list and compute in-degree.
2. Push all nodes with in-degree 0 into a queue.
3. Process nodes: pop, add to order, decrement neighbour in-degrees.
4. If `processed < total_nodes`, a cycle exists.

DFS Approach (Three-Colour):

1. Create `vis` array: 0 = unvisited, 1 = in stack, 2 = done.
2. For each unvisited node, call `dfs(u)`.
3. In `dfs(u)`:
 - Mark `vis[u] = 1`.
 - For each neighbour `v`:
 - If `vis[v] == 1` → cycle found.
 - If `vis[v] == 0` → recurse.
 - Mark `vis[u] = 2`.
4. If any DFS call returns cycle, the graph has a cycle.

7. Dry Run

Graph with cycle: $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$

Kahn's approach:

Step	Queue	Processed	indeg[0]	indeg[1]	indeg[2]
Init	[]	0	1	1	1
1	(empty)	0	1	1	1

No node has in-degree 0. Queue is empty. Processed = 0 < 3 → cycle detected. ✓

DFS approach (starting from 0):

Step	Node	vis[0]	vis[1]	vis[2]	Action
Init	-	0	0	0	-
1	0	1	0	0	Mark grey, go to 1
2	1	1	1	0	Mark grey, go to 2
3	2	1	1	1	Mark grey, neighbour 0 is grey → CYCLE

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class DirectedCycleDetector {
```

```

public:
    // Method 1: Kahn's algorithm (BFS-based)
    bool hasCycleKahn(int n, vector<vector<int>> &adj) {
        vector<int> indeg(n, 0);
        for (int u = 0; u < n; u++) {
            for (int v : adj[u]) {
                indeg[v]++;
            }
        }

        queue<int> q;
        for (int i = 0; i < n; i++) {
            if (indeg[i] == 0) q.push(i);
        }

        int processed = 0;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            processed++;
            for (int v : adj[u]) {
                indeg[v]--;
                if (indeg[v] == 0) q.push(v);
            }
        }

        return processed != n; // cycle exists if not all processed
    }

    // Method 2: DFS with three-colour marking
    bool hasCycleDFS(int n, vector<vector<int>> &adj) {
        vector<int> vis(n, 0); // 0=white, 1=grey, 2=black
        bool hasCycle = false;

        function<void(int)> dfs = [&](int u) {
            vis[u] = 1; // grey
            for (int v : adj[u]) {
                if (vis[v] == 1) {
                    hasCycle = true;
                    return;
                }
                if (vis[v] == 0) {
                    dfs(v);
                    if (hasCycle) return;
                }
            }
            vis[u] = 2; // black
        };

        for (int i = 0; i < n; i++) {
            if (vis[i] == 0) {
                dfs(i);
                if (hasCycle) return true;
            }
        }
        return false;
    }

};

int main() {

```

```

int n = 3;
vector<vector<int>> adj(n);
adj[0] = {1};
adj[1] = {2};
adj[2] = {0}; // creates cycle: 0 → 1 → 2 → 0

DirectedCycleDetector dcd;
cout << "Kahn: " << (dcd.hasCycleKahn(n, adj) ? "Cycle" : "No cycle") << "\n";
cout << "DFS: " << (dcd.hasCycleDFS(n, adj) ? "Cycle" : "No cycle") << "\n";

// DAG test
vector<vector<int>> dag(3);
dag[0] = {1, 2};
dag[1] = {2};
cout << "Kahn (DAG): " << (dcd.hasCycleKahn(3, dag) ? "Cycle" : "No cycle") << "\n";
cout << "DFS (DAG): " << (dcd.hasCycleDFS(3, dag) ? "Cycle" : "No cycle") << "\n";

return 0;
}

```

Output:

```

Kahn: Cycle
DFS: Cycle
Kahn (DAG): No cycle
DFS (DAG): No cycle

```

Python Implementation

```

from collections import deque
from typing import List

class DirectedCycleDetector:
    @staticmethod
    def has_cycle_kahn(n: int, adj: List[List[int]]) -> bool:
        indeg = [0] * n
        for u in range(n):
            for v in adj[u]:
                indeg[v] += 1

        q = deque([i for i in range(n) if indeg[i] == 0])
        processed = 0

        while q:
            u = q.popleft()
            processed += 1
            for v in adj[u]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    q.append(v)

        return processed != n

    @staticmethod
    def has_cycle_dfs(n: int, adj: List[List[int]]) -> bool:

```

```

vis = [0] * n # 0=white, 1=grey, 2=black
has_cycle = False

def dfs(u: int) -> None:
    nonlocal has_cycle
    vis[u] = 1
    for v in adj[u]:
        if vis[v] == 1:
            has_cycle = True
            return
        if vis[v] == 0:
            dfs(v)
            if has_cycle:
                return
    vis[u] = 2

for i in range(n):
    if vis[i] == 0:
        dfs(i)
        if has_cycle:
            return True
return False

if __name__ == "__main__":
    n = 3
    adj = [[1], [2], [0]]
    print("Kahn:", "Cycle" if DirectedCycleDetector.has_cycle_kahn(n, adj) else "No cycle")
    print("DFS:", "Cycle" if DirectedCycleDetector.has_cycle_dfs(n, adj) else "No cycle")

    dag = [[1, 2], [2], []]
    print("Kahn (DAG):", "Cycle" if DirectedCycleDetector.has_cycle_kahn(3, dag) else "No
cycle")
    print("DFS (DAG):", "Cycle" if DirectedCycleDetector.has_cycle_dfs(3, dag) else "No
cycle")

```

10. Code Explanation

Kahn's approach:

- Compute in-degree for all nodes.
- Push nodes with in-degree 0 into a queue.
- Process nodes, decrement neighbour in-degrees.
- If `processed != n`, some nodes are in a cycle.
- **Why it works:** In a cycle, every node has at least one incoming edge from within the cycle. These edges never get removed because the source nodes are never processed.

DFS approach:

- Three-colour marking: white (0) = unvisited, grey (1) = in current DFS path, black (2) = processed.
- When we encounter a grey neighbour, we've found a back edge → cycle.
- **Why it works:** In a directed graph, a cycle is exactly a path from a node back to itself. The grey state tracks nodes currently on the recursion stack. If we find an edge to a node on the stack, we've found a cycle.

11. Complexity Analysis

Method	Time	Space
Kahn's (cycle detection)	$O(V + E)$	$O(V)$
DFS (three-colour)	$O(V + E)$	$O(V)$ for recursion stack + vis array

12. Common Patterns

12.1 Cycle Detection Only

- **How to identify:** "Does the graph have a cycle?"
- **Approach:** Either Kahn's or DFS. Both are $O(V+E)$.
- **Example:** [LeetCode 207](#) — [Course Schedule](#)

12.2 Find All Nodes in Cycles

- **How to identify:** "Find all nodes that are part of a cycle."
- **Approach:** Run Kahn's. Nodes not processed ($\text{indeg} > 0$ at end) are in a cycle.
- **Example:** [Codeforces](#) — [Cycle in a Graph](#)

12.3 Minimum Edges to Remove to Make DAG

- **How to identify:** "Minimum edges to remove to make graph acyclic."
- **Approach:** Find all cycles (Kahn's), count edges in cycles.
- **Example:** [GFG](#) — [Minimum edges to remove to make graph acyclic](#)

13. Common Mistakes

- **Using only visited/unvisited for DFS:** Without the grey state, you can't distinguish between an edge to a previously processed node (forward/cross edge) and an edge to an ancestor (back edge).
- **Not checking all components:** A cycle might be in a disconnected component. Always iterate over all nodes.
- **Assuming Kahn's is only for topological sort:** Kahn's is excellent for cycle detection too.
- **Forgetting self-loops:** A self-loop $u \rightarrow u$ is a cycle. $\text{indeg}[u]$ will be at least 1, and the node will never be processed.

14. Edge Cases

- **Self-loop:** $\text{adj}[u] = [u] \rightarrow$ cycle detected immediately.
- **Two-node cycle:** $0 \rightarrow 1 \rightarrow 0 \rightarrow$ cycle detected.
- **Disconnected graph with one cyclic component:** Cycle detected.
- **Empty graph:** No cycle.
- **Single node, no edges:** No cycle.
- **DAG with multiple components:** No cycle.

15. Variations

15.1 Find One Cycle in the Graph

- Instead of just detecting, return the nodes in the cycle.
- **Approach:** Use DFS with parent tracking. When a back edge is found, trace back using parent pointers.
- **Placement relevance:** Moderate.

15.2 Find All Cycles

- **Approach:** Use Johnson's algorithm or Tarjan's algorithm for strongly connected components.
- **Placement relevance:** Low for placements. More theoretical.

16. Related Algorithms/Data Structures

- **DFS with three-colour marking:** The standard approach for directed cycle detection.
- **Kahn's algorithm:** Simpler, avoids recursion, but doesn't give cycle nodes directly.
- **Union-Find (DSU):** Used for undirected cycle detection, not directed.
- **Kosaraju/Tarjan (SCC):** Finds all strongly connected components. A cycle is an SCC with more than one node (or a self-loop).

5. Longest Path in DAG

1. Overview

The longest path problem in a general graph is NP-hard. However, in a **Directed Acyclic Graph (DAG)**, the longest path can be found in $O(V + E)$ time using topological sort and dynamic programming.

This is a classic placement and CP problem that demonstrates the power of combining topological ordering with DP.

2. Intuition

Imagine planning a **project with tasks** where each task has a duration, and some tasks depend on others. You want to know the **minimum time to complete the project** (the critical path). This is the longest path in the dependency DAG.

The approach:

- First, get a topological order of the DAG. This ensures we process nodes in dependency order.
- When processing a node u , we relax all its outgoing edges: $\text{dist}[v] = \max(\text{dist}[v], \text{dist}[u] + \text{weight}(u, v))$.
- Since we process nodes in topological order, when we process u , all paths to u have already been considered.

Why it works: In a topological order, when we process node u , all paths from sources to u have already been explored. So $\text{dist}[u]$ is the final longest distance to u . We then propagate this to u 's neighbours.

3. When to Use It

- **Critical path analysis** in project management
- **Minimum time to complete all tasks** (parallel execution)
- **Maximum cost path** in a DAG
- **Longest chain of dependencies**
- **Finding the longest increasing subsequence** in a DAG formulation
- **Trigger phrases:** "longest path", "critical path", "minimum time to complete", "maximum cost", "longest chain", "longest sequence"

4. When Not to Use It

- **Graph has cycles** — the problem becomes NP-hard (use Bellman-Ford with negative weights for shortest path, but longest path is NP-hard in general graphs)
- **Graph is undirected** — longest path in undirected graph is NP-hard
- **Need shortest path** — use BFS (unweighted), Dijkstra (positive weights), or Bellman-Ford (negative weights)
- **Overkill:** For a simple linear chain, just traverse

5. Core Concepts

5.1 Topological Order

- The foundation for the DP. We process nodes in topological order to ensure all dependencies are resolved.

- **Why it matters:** Without topological order, we'd need to revisit nodes multiple times.

5.2 DP Array (dist)

- `dist[u]` = longest distance from a source to node `u`.
- **Why it matters:** This stores the best result for each node, updated as we process.

5.3 Relaxation

- For each edge `u → v` with weight `w`, we check: `dist[v] = max(dist[v], dist[u] + w)`.
- **Why it matters:** This is how we propagate distances forward.

5.4 Source Nodes

- Nodes with in-degree 0. They have no incoming dependencies.
- **Why it matters:** These are the starting points. `dist[source] = 0` (or weight of source node itself).

6. Step-by-Step Algorithm

1. Build adjacency list with edge weights.
2. Compute topological order using Kahn's algorithm or DFS.
3. Initialize `dist` array with `-INF` (or `-1` for unreachable).
4. For each source node (in-degree 0), set `dist[source] = 0`.
5. Process nodes in topological order:
 - For each neighbour `v` of `u` with weight `w`:
 - If `dist[u] != -INF` and `dist[u] + w > dist[v]`:
 - Update `dist[v] = dist[u] + w`.
6. The longest path is `max(dist)` over all nodes.

For longest path ending at each node:

- Initialize `dist[i] = 0` for all nodes.
- Process in topological order: `dist[v] = max(dist[v], dist[u] + w)`.
- Result is `max(dist)`.

7. Dry Run

Graph:

```

0 →(5) 1 →(3) 3
↓(2)      ↑
2 →(4) ———┘

```

Edges (weighted): `0→1(5)`, `0→2(2)`, `1→3(3)`, `2→3(4)`

Topological order: `[0, 1, 2, 3]`

Step	Node	dist[0]	dist[1]	dist[2]	dist[3]	Action
Init	-	0	-INF	-INF	-INF	Sources: 0
1	0	0	5	2	-INF	<code>dist[1]=max(-INF,0+5)=5</code> , <code>dist[2]=max(-INF,0+2)=2</code>
2	1	0	5	2	8	<code>dist[3]=max(-INF,5+3)=8</code>
3	2	0	5	2	8	<code>dist[3]=max(8,2+4)=8</code> (no change)

Step	Node	dist[0]	dist[1]	dist[2]	dist[3]	Action
4	3	0	5	2	8	No outgoing edges

Longest path: 8 (path: 0 → 1 → 3)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class LongestPathDAG {
public:
    // Returns the longest path distance from any source to any destination.
    // If graph has a cycle, returns -1 (impossible for DAG).
    int longestPath(int n, vector<vector<pair<int,int>>> &adj) {
        // Step 1: Compute in-degree for topological sort
        vector<int> indeg(n, 0);
        for (int u = 0; u < n; u++) {
            for (auto &[v, w] : adj[u]) {
                indeg[v]++;
            }
        }

        // Step 2: Kahn's algorithm to get topological order
        queue<int> q;
        for (int i = 0; i < n; i++) {
            if (indeg[i] == 0) q.push(i);
        }

        vector<int> topo;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            topo.push_back(u);
            for (auto &[v, w] : adj[u]) {
                indeg[v]--;
                if (indeg[v] == 0) q.push(v);
            }
        }

        // If graph has a cycle, topological sort fails
        if ((int)topo.size() != n) return -1;

        // Step 3: DP for longest path
        vector<int> dist(n, INT_MIN);
        for (int u : topo) {
            if (dist[u] == INT_MIN) dist[u] = 0; // source node
            for (auto &[v, w] : adj[u]) {
                if (dist[u] != INT_MIN) {
                    dist[v] = max(dist[v], dist[u] + w);
                }
            }
        }

        // Step 4: Find maximum distance
        int ans = 0;
        for (int d : dist) {
```

```

        if (d != INT_MIN) ans = max(ans, d);
    }
    return ans;
}

// Returns the longest path ending at each node (starting from any source)
vector<int> longestPathEndingAtEachNode(int n, vector<vector<pair<int,int>>> &adj) {
    // Same topological sort as above
    // ... (omitted for brevity, same as above)
    return {};
}

};

int main() {
    int n = 4;
    vector<vector<pair<int,int>>> adj(n);
    adj[0] = {{1, 5}, {2, 2}};
    adj[1] = {{3, 3}};
    adj[2] = {{3, 4}};
    // No outgoing edges from 3

    LongestPathDAG solver;
    int result = solver.longestPath(n, adj);
    cout << "Longest path in DAG: " << result << "\n";

    return 0;
}

```

Output:

```
Longest path in DAG: 8
```

Python Implementation

```

from collections import deque
from typing import List, Tuple

class LongestPathDAG:
    @staticmethod
    def longest_path(n: int, adj: List[List[Tuple[int, int]]]) -> int:
        # Step 1: In-degree for topological sort
        indeg = [0] * n
        for u in range(n):
            for v, w in adj[u]:
                indeg[v] += 1

        # Step 2: Kahn's algorithm
        q = deque([i for i in range(n) if indeg[i] == 0])
        topo = []

        while q:
            u = q.popleft()
            topo.append(u)
            for v, w in adj[u]:
                indeg[v] -= 1

```

```

        if indeg[v] == 0:
            q.append(v)

    if len(topo) != n:
        return -1 # cycle detected

    # Step 3: DP for longest path
    dist = [float('-inf')] * n
    for u in topo:
        if dist[u] == float('-inf'):
            dist[u] = 0 # source node
        for v, w in adj[u]:
            if dist[u] != float('-inf'):
                dist[v] = max(dist[v], dist[u] + w)

    # Step 4: Find maximum
    ans = 0
    for d in dist:
        if d != float('-inf'):
            ans = max(ans, d)
    return ans

if __name__ == "__main__":
    n = 4
    adj = [[] for _ in range(n)]
    adj[0] = [(1, 5), (2, 2)]
    adj[1] = [(3, 3)]
    adj[2] = [(3, 4)]

    result = LongestPathDAG.longest_path(n, adj)
    print("Longest path in DAG:", result)

```

10. Code Explanation

- Topological sort:** We use Kahn's algorithm. This gives us a processing order where all incoming edges to a node are from nodes that appear earlier in the order.
- DP initialization:** `dist` is initialized to `INT_MIN` (or `-inf`). Source nodes (in-degree 0) are set to `dist[source] = 0`.
- DP relaxation:** For each node `u` in topological order, we look at all outgoing edges `u → v` with weight `w`. We update `dist[v] = max(dist[v], dist[u] + w)`. This is analogous to the relaxation step in shortest path algorithms, but with `max` instead of `min`.
- Why topological order matters:** When we process `u`, we've already processed all nodes that have edges to `u`. So `dist[u]` is the final longest distance to `u`. There's no need to revisit.
- Result:** The maximum value in `dist` is the longest path in the DAG.

11. Complexity Analysis

Step	Time	Space
Topological sort	$O(V + E)$	$O(V)$
DP relaxation	$O(V + E)$	$O(V)$
Overall	$O(V + E)$	$O(V)$

12. Common Patterns

12.1 Longest Path from Any Source to Any Destination

- **How to identify:** "Longest path in DAG", "maximum cost path"
- **Approach:** Topological sort + DP with `max` relaxation.
- **Example:** [GFG — Longest Path in a DAG](#)

12.2 Longest Path from a Specific Source

- **How to identify:** "Longest path from node X"
- **Approach:** Set `dist[source] = 0`, others = `-INF`. Process in topological order.
- **Example:** [SPOJ — LONGEST PATH IN DAG](#)

12.3 Minimum Time to Complete All Tasks (Critical Path)

- **How to identify:** "Minimum time to finish all tasks", "parallel execution"
- **Approach:** Longest path in the dependency DAG. This is the critical path length.
- **Example:** [LeetCode 2050 — Parallel Courses III](#)

12.4 Longest Path in a Grid (DAG Formulation)

- **How to identify:** "Longest increasing path in a matrix"
- **Approach:** Each cell is a node, edges go from smaller to larger values. This is a DAG. Use topological sort + DP.
- **Example:** [LeetCode 329 — Longest Increasing Path in a Matrix](#)

13. Common Mistakes

- **Not handling unreachable nodes:** Some nodes may not be reachable from any source. Their `dist` remains `-INF`. Exclude them from the answer.
- **Wrong initialization:** Initializing `dist` to 0 for all nodes treats all nodes as sources. Only sources should have 0.
- **Forgetting cycle detection:** If the graph has a cycle, the longest path is not defined. Always check for cycles first.
- **Using `-1` as unreachable:** If edge weights can be negative, `-1` might be a valid distance. Use `INT_MIN` or `-inf`.
- **Not processing in topological order:** Processing nodes in arbitrary order may give wrong results because a node's neighbours might be processed before the node itself.

14. Edge Cases

- **Empty graph ($n=0$):** Return 0.
- **Single node, no edges:** Return 0 (path of zero length).
- **All nodes disconnected:** Return 0 (no edges means path length 0).
- **Negative weights:** The algorithm works with negative weights (unlike Dijkstra). But be careful with initialization.
- **Multiple sources:** All sources should have `dist = 0`.
- **Graph with in-degree 0 for all nodes:** No edges. All nodes are sources. Answer is 0.
- **Linear chain ($1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow n$):** Longest path is the sum of all edge weights.

15. Variations

15.1 Longest Path in a Weighted DAG (Node Weights)

- Each node has a weight, not edges.
- **Approach:** `dist[v] = max(dist[v], dist[u] + weight[v])` when processing edge $u \rightarrow v$.
- **Placement relevance:** High. Common in problems where each course/task has a duration.

15.2 Number of Longest Paths

- Count how many different paths achieve the longest distance.
- **Approach:** Maintain a `count[]` array alongside `dist[]`. When updating, reset or increment count.
- **Placement relevance:** Moderate.

15.3 Longest Path with Constraints

- Additional constraints like "you can only take at most k courses per semester".
- **Approach:** Combine topological sort with DP and greedy scheduling.
- **Placement relevance:** High. Common in OAs.

16. Related Algorithms/Data Structures

- **Shortest Path in DAG:** Same approach but with `min` instead of `max` and `dist[u] + w < dist[v]`.
- **Dijkstra's Algorithm:** For shortest path in general weighted graphs (non-negative weights). Not applicable to longest path.
- **Bellman-Ford:** For shortest path with negative weights. Can detect negative cycles.
- **Floyd-Warshall:** All-pairs shortest path. Not efficient for longest path.
- **DP on DAG:** The general pattern of topological sort + DP applies to many problems (count paths, min/max path, etc.).

Practice Problems

Easy

1. Course Schedule

- **Platform:** LeetCode 207
- **Main idea:** Detect if a directed graph has a cycle using Kahn's algorithm.
- **Difficulty:** Easy

2. Course Schedule II

- **Platform:** LeetCode 210
- **Main idea:** Return a topological order of courses using Kahn's algorithm.
- **Difficulty:** Easy

Medium

1. Find Eventual Safe States

- **Platform:** LeetCode 802
- **Main idea:** Nodes that are not part of any cycle. Use reverse graph + Kahn's (topological sort on reversed graph).
- **Difficulty:** Medium

2. Alien Dictionary

- **Platform:** LeetCode 269 (Premium) / GFG
- **Main idea:** Extract character ordering from a sorted dictionary, then topological sort to find the order of letters.
- **Difficulty:** Medium

3. Parallel Courses III

- **Platform:** LeetCode 2050
- **Main idea:** Longest path in DAG with node weights (course durations). Topological sort + DP.

- **Difficulty:** Medium

Hard

1. Longest Increasing Path in a Matrix

- **Platform:** LeetCode 329
- **Main idea:** Treat the matrix as a DAG (edges from smaller to larger values). Use DFS with memoization or topological sort + DP.
- **Difficulty:** Hard

2. Minimum Height Trees

- **Platform:** LeetCode 310
- **Main idea:** Topological sort-like approach (removing leaves layer by layer). Find the centroids of a tree.
- **Difficulty:** Hard

3. Sort Items by Groups Respecting Dependencies

- **Platform:** LeetCode 1203
- **Main idea:** Two-level topological sort (items within groups, and groups themselves). Complex graph construction.
- **Difficulty:** Hard

Interview Explanation

Here's what to say when asked about topological sort in an interview:

"Topological sort is a linear ordering of vertices in a directed acyclic graph such that for every edge $u \rightarrow v$, u comes before v . It's used for dependency resolution, course scheduling, and build systems.

There are two main approaches: **DFS-based** and **Kahn's algorithm** (BFS-based).

DFS approach: We perform DFS and add nodes to a stack after processing all their neighbours. The stack, read in reverse, gives the topological order. We use three-colour marking (white/grey/black) to detect cycles — if we encounter a grey node, we've found a back edge.

Kahn's algorithm: We compute in-degrees, push nodes with in-degree 0 into a queue, and repeatedly remove them while decrementing neighbour in-degrees. If a neighbour's in-degree reaches 0, it's added to the queue. If the number of processed nodes doesn't equal the total, a cycle exists.

Both run in $O(V+E)$ time and $O(V)$ space. Kahn's is more intuitive for cycle detection and easily modified for lexicographically smallest order by using a min-heap. The DFS approach is simpler to code but risks stack overflow on very deep graphs.

A powerful extension is the **longest path in a DAG** — we compute topological order, then run DP: $\text{dist}[v] = \max(\text{dist}[v], \text{dist}[u] + \text{weight}(u, v))$. This is $O(V+E)$ and solves the critical path problem, which is NP-hard in general graphs."

Revision Notes

- **Topological sort:** Linear ordering of DAG where all edges go from earlier to later nodes.
- **DFS approach:** Post-order insertion + reverse. Three-colour for cycle detection.
- **Kahn's approach:** In-degree queue. Processed count $< n \rightarrow$ cycle.

- **Longest path in DAG:** Topo order + DP with **max** relaxation. $O(V+E)$.
- **Complexity:** $O(V+E)$ time, $O(V)$ space.
- **Common traps:**
 - Wrong edge direction (dependencies vs dependents)
 - Forgetting to reverse DFS result
 - Using two states instead of three for cycle detection
 - Not handling disconnected graphs
 - Not checking processed count in Kahn's
- **Must-know problems:** Course Schedule I & II, Alien Dictionary, Longest Path in DAG, Parallel Courses III

Final Cheat Sheet

Aspect	DFS-Based	Kahn's Algorithm
When to use	Any DAG ordering, cycle detection	Same + lexicographically smallest order
Core idea	Post-order DFS + reverse	In-degree queue + source removal
Cycle detection	Back edge (grey node)	processed < n
Time	$O(V+E)$	$O(V+E)$
Space	$O(V)$	$O(V)$
Key code	<code>vis[u]=1; for v: if(vis[v]==1) cycle; dfs(v); vis[u]=2; order.push(u)</code>	<code>indeg[v]++; q.push(indeg 0); while(!q.empty()) {u=q.pop(); for v: indeg[v]--; if(indeg[v]==0) q.push(v)}</code>
Edge cases	Self-loop, disconnected graph, empty graph	Same + all nodes in cycle
Variation	Longest Path in DAG	Lexicographically smallest order

Longest Path in DAG Code Template:

```
// 1. Topological sort (Kahn's)
// 2. DP: dist[v] = max(dist[v], dist[u] + weight(u, v))
// 3. Answer = max(dist)
```

Course Schedule Template:

```
// 1. Build adj: for [a,b]: adj[b].push_back(a); indeg[a]++;
// 2. Kahn's: queue of indeg 0 nodes
// 3. If processed == n → possible, return order
// 4. Else → impossible, return empty
```